

PROJECT 1 – Document Classification

Important Notes:

1. If you do not have time to finish the project during the practical sessions, **you must finish it later.**
2. **Must be used:**
 - a) **The same name we propose for the functions and procedures**
 - b) **The same datatypes we propose**
 - c) **The same output messages**

Otherwise, the correction system will not work, and you will not pass the evaluation.

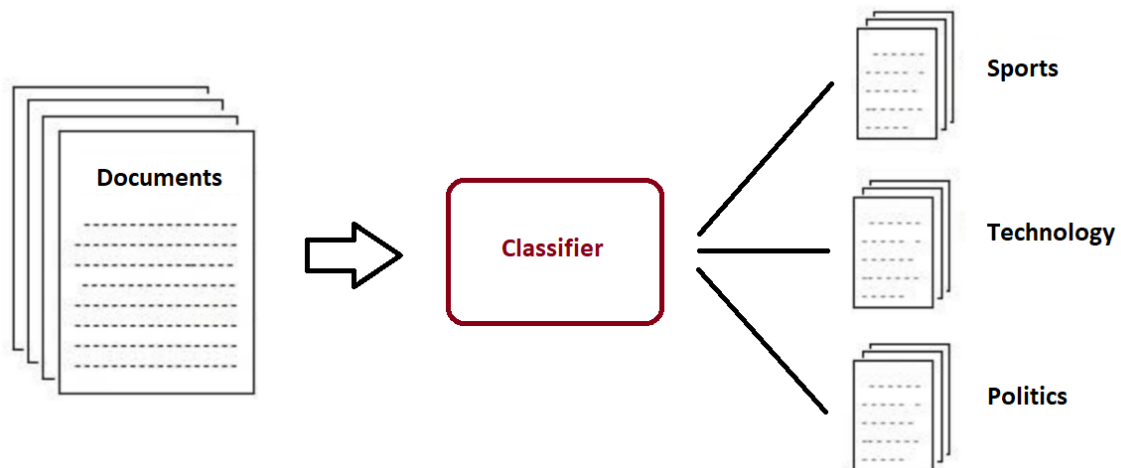
Introduction

Document classification is the process of assigning categories to documents to make them easier to search, sort, analyze, filter, etc. A document contains information related to some specific category (or topics). Assigning one or more classes (categories/topics) to a document makes it extremely beneficial for news sites, blogs, business industries, magazine publishers or anybody dealing with a huge amount of content.

Document classification is seen as a very useful Machine Learning (ML) application in the domain of Natural Language Processing (NLP). In general, there are two types of ML techniques: supervised and unsupervised. A ML model is built using **supervised** methods by creating a **training set**, which contains predefined categories (called **classes**) of documents and every document is tagged with one or more categories (called **labels**). A document **classifier** ML model is then trained on the dataset to perform prediction of a new document's category during the later **test** phase. The classifier may also additionally offer a confidence score to show how confident it is that the classification label is correct, depending on the classification algorithm or strategy utilized.

An example illustration of a Document Classification Model

Intuitively, given that a document is about a particular topic, one would expect some frequent words. For example, "goal" and "corner" will often appear in documents about football, "president" and "ministry" will appear in documents about politics, whereas stop words, like "the" and "with" will appear approximately equally in both. Indeed, words in a document can be used as a powerful tool (called **features**) to help in the prediction of a document category.



Goal:

The project consists of developing a program to classify documents. You will be provided with a **News dataset collection** containing documents in **txt** format of different categories (e.g. business, medical, space, sports, politics, etc.), and a **txt** file with the list of keywords per category.

Exercise 1: Implement *DocClass_Welcome()*

Implement a procedure called *DocClass_Welcome()*. This procedure has no input parameters and must display the following informational message:

```
Document Classification is typically used in Document Processing for
assigning categories to documents. Thus, it eases their posterior
processing and analysis.
```

```
The program "DocumentClassification" classifies an input folder
containing documents in txt format. It will also receive a file
containing information about the topics to classify.
```

```
The program will generate the following files:
A txt file containing the category of the documents
A txt file containing the statistics of the classification
```

Exercise 2: Implement *Read_Categories_Keywords()*

The function *Read_Categories_Keywords()* will open the **txt** file and read the categories and the keywords of each category. The input will be the name of the file containing the lists of categories and keywords (e.g. **keywords.txt**), whereas the output will be:

- *LCategories*: list of categories (*strings*).
- *LKeywords*: list of keywords per category (list of *sets*). Each element in the list consists of a *set*, with the keywords in that category.

The format of the input file (e.g. **keywords.txt**) will consist of one category per line, with keywords separated by blank space. For example:

```
Category1: KeyWord1A KeyWord1B KeyWord1C
Category2: KeyWord2A KeyWord2B KeyWord2C KeyWord2D KeyWord2E
...
CategoryN: KeyWordNA KeyWordNB KeyWordNC KeyWordND
```

Important: The number of categories and the number of keywords per category are **variable**.

For example, if the list of categories and keywords are:

```
medical: physician infection patient care
space: planet orbit nasa
sports: medal record score team training
```

The output should be:

- *LCategories* = ["medical", "space", "sports"]
- *LKeywords* = [{"physician", "infection", "patient", "care"}, {"planet", "orbit", "nasa"}, {"medal", "record", "score", "team", "training"}]

Exercise 3: Implement *Document_Reader()*

The function *Document_Reader()* receives, as input, the path, the folder name and the filename as parameters. It opens the file and reads its contents. It returns the list of words *LWords* appearing in the document file.

Important:

recommended to convert to lower case

- This function is **case insensitive**, which means that lower and uppercase letters are equivalent (NASA = nasa, Federer = federer).
- Apostrophes are joined to the word. For example, "don't" is considered one word.
- Numbers are considered as one word. For example, 19.
- All the rest of characters are considered as word separators, for example: Full Stop (.), Question Mark (?), Quotation Marks ("), Comma (,), Hyphen (-), dash (–), Exclamation Mark (!), Colon (:), Semicolon (;), Parentheses (), Brackets [], Ellipsis (...), Slash (/), etc.

For example, if a filename contains the text:

"Once rocket 12 reached the planet, the NASA-team celebrated a party!
My 15-year-old daughter couldn't (barely) sleep."

LWords will be: ["once", "rocket", "12", "reached", "the", "planet", "the", "nasa", "team", "celebrated", "a", "party", "my", "15", "year", "old", "daughter", "couldn't", "barely", "sleep"].

Exercise 4: Implement *Count_Frequencies()*

Create a function called *Count_Frequencies()* that takes as input the following parameters:

- *LWords*: list of words in the document.
- *LCategories*: list of categories.
- *LKeywords*: list of keywords in each category.

The function will output:

- *DFrequencies*: Dictionary (nested Dictionary) with the number of occurrences of each keyword per category.

The function will count the number of occurrences of each keyword per category in the given input file and return the list *DFrequencies*. For example, if the lists of categories and keywords are:

```
medical: physician infection patient care
space: planet orbit nasa
sports: medal record score team training
```

and the document file contains the text:

```
"Once rocket 12 reached the right distance from the planet, the
NASA-team celebrated a big party at the nasa main building."
```

The output list *DFrequencies* should be:

- {'medical': {'physician': 0, 'infection': 0, 'patient': 0, 'care': 0},
'space': {'planet': 1, 'orbit': 0, 'nasa': 2},
'sports': {'medal': 0, 'record': 0, 'score': 0, 'team': 1, 'training': 0}
}

Exercise 5: Implement *Classify_Doc()*:

Create a function called *Classify_Doc()* that takes as input the dictionary of frequencies *DFrequencies* and outputs:

- *DVotes*: Dictionary of total occurrences and total percentage in each category: The key will be the keyword, whereas the value will be a *tuple* of *integers*, containing (TotalNumOccurrences, TotalPercentage).
- *DocCategory*: the category of the document (*string*).

This function should count the total number of keyword occurrences in each category. Then, it will compute the total percentage (value between 0-100%) that will be used to classify the document. For example, if *DFrequencies* is

- {'medical': {'physician': 0, 'infection': 0, 'patient': 0, 'care': 0},
 'space': {'planet': 1, 'orbit': 0, 'nasa': 2},
 'sports': {'medal': 0, 'record': 0, 'score': 0, 'team': 1, 'training': 0}
 }

watch out for division by zero

the output will be (3 occurrences of category 'space', so 75% and 1 occurrence of 'sports', so 25%):

- *DVotes* = {'medical': (0,0), 'space': (3,75), 'sports': (1,25)}

Finally, the function will find the position of the category with the highest percentage of occurrences (in other words, the maximum percentage value in *DVotes*). According to the previous example, the output will be:

- *DocCategory* = "space".

IMPORTANT: For the final classification, a **percentage higher than 50%** is necessary. Otherwise, the document will be classified as UNKNOWN. For example, in case some documents have *DVotes* = {'medical': (0,0), 'space': (2,50), 'sports': (2,50)} ('space' and 'sports' have both 50%) or *DVotes* = {'medical': (2,40), 'space': (1,20), 'sports': (1,20), 'politics': (1,20)} ('medical' has 40%), then, the output will be, in both cases:

- *DocCategory* = "UNKNOWN".

Exercise 6: Implement *main()*:

Implement the main program that will make the calls to the corresponding functions and procedures to classify the documents. The sequence is as follows:

1. Call the procedure *DocClass_Welcome()* to inform about the program.
2. Ask the name of the file that contains the definition of the categories and keywords. If the file does not exist or contains no data, ask the user again, until a valid file is entered. The file must be in the same directory of your python script.
3. Read the contents of the file, using the function *Read_Categories_Keywords()*.
4. Ask the name of the folder that contains the document files to be processed. If the folder does not exist or contains no *txt* files, ask the user again, until a valid folder name is entered. The folder must be in the same directory of your python script.
5. For each file in the folder, do the following:
 - a) If it is a **txt** file, call the functions *Document_Reader()*, *Count_Frequencies()* and *Classify_Doc()*. In the file named "<folder_name>_DocClassification.txt", write the category of each **txt** file, with the percentages per category, and the number of occurrences of each keyword. In case a file has category UNKNOWN, do not write the keywords. Use the following format:

```
Filename1: Category3(C1:P1, C2:P2...): Keyword3A:X, Keyword3B:Y, ...  
Filename2: Category1(C1:P1, C2:P2...): Keyword1A:X, Keyword1B:Y, ...  
Filename3: UNKNOWN (C1:P1, C2:P2...)  
...  
FilenameM: Category5(C1:P1, C2:P2...): Keyword5A: X, Keyword5B: Y, ...
```

- b) If the file has any other format, it should write the message: “Attention: the following non txt file has been detected: <filename>” in the file “<folder_name>_DocClassification.txt” (same as above). For example:

```
news1.txt: sports(medical:15%, space:9.6%, sports:75.4%): medal:1,  
record:3, score:1, participate:0, world:1 ...  
news2.txt: UNKNOWN (medical:20%, space:50%, sports:30%)  
...  
Attention: the following non txt file has been detected: news34.doc  
...  
news39.txt: space(medical:49%, space:51%, sports:0%): space:1,  
launch:4, complex:2, ...
```

The use of **os.path.join()** to create the path of the files that are inside the folder is **mandatory**. Use it as follows:

- `file_path = os.path.join(folder_name, file_n_name)`
6. Once all files in the folder are classified, compute the percentage (maximum 2 decimals) of **txt** documents of each category (note that for the statistics, the number of non-txt files does not count). Write the result in a file called “<folder_name>_DCStatistics.txt” (for example, if the folder is “MyDocs”, the output file will be “MyDocs_DCStatistics.txt”). Also, compute the top 10 keywords in the folder (no matter if they belong to different categories) and the number of occurrences of each one. The format of the file will be:

```
*** Statistics  
Total txt files: N  
Category1: XX.XX%  
Category2: YY.YY%  
...  
CategoryN: ZZ%  
  
*** Top 10 keywords  
KeyWord1: NumOccurrencesKeyWord1  
KeyWord2: NumOccurrencesKeyWord2  
...  
KeyWord10: NumOccurrencesKeyWord10
```

Important Remarks:

- Writing the main program should be under the **main guard**, while the functions are above the main guard, as follows:

```
def function_n(param_1,..., param_m):  
    # the function code goes here  
    return out_1, ..., out_p  
  
if __name__ == "__main__":  
    # the main code goes here  
    out1, outp = function_n(param1, .., param_p)
```

- The management of possible errors (e.g. file does not exist, division by zero) is mandatory. For example:

```
try:  
    f = open(filename, "r")  
except:  
    raise FileNotFoundError
```

- Feel free to create more functions if necessary.
- **Comments** in the code are **mandatory**. Add comments to explain your code.

Hints

Some useful instructions for reading all files in a folder:

The OS module in Python provides functions for interacting with the operating system. If you import this module, then:

- The function **os.listdir()** returns the list of all files and folders in the specified path of the directory.
- You can use **os.path.exists()** to check if files and/or directories exist. It returns a Boolean.
- You can use **os.path.join()** to create the desired paths of the files that you will read.

More info at <https://www.geeksforgeeks.org/os-module-python-examples/>

Note. In some cases, reading some files could be problematic due to a different file encoding. In such a case, try different encodings when opening the file. For example:

- `file = open(filename, "r", encoding='mac_roman')`
- `file = open(filename, "r", encoding=encoding='windows-1252')`

Delivery

The project will be done in pairs, and the delivery will be via *Caronte*. Instructions:

- Upload the file "DocumentClassification.py".
- Only **one of the members** of the group must do it.
- At the beginning of this Python file, write, **as comments**, the name, surname and NIU of the members of the group.

Deadline: **December 8th, 2024.**